Advanced Programming

Noel O'Hara PhD

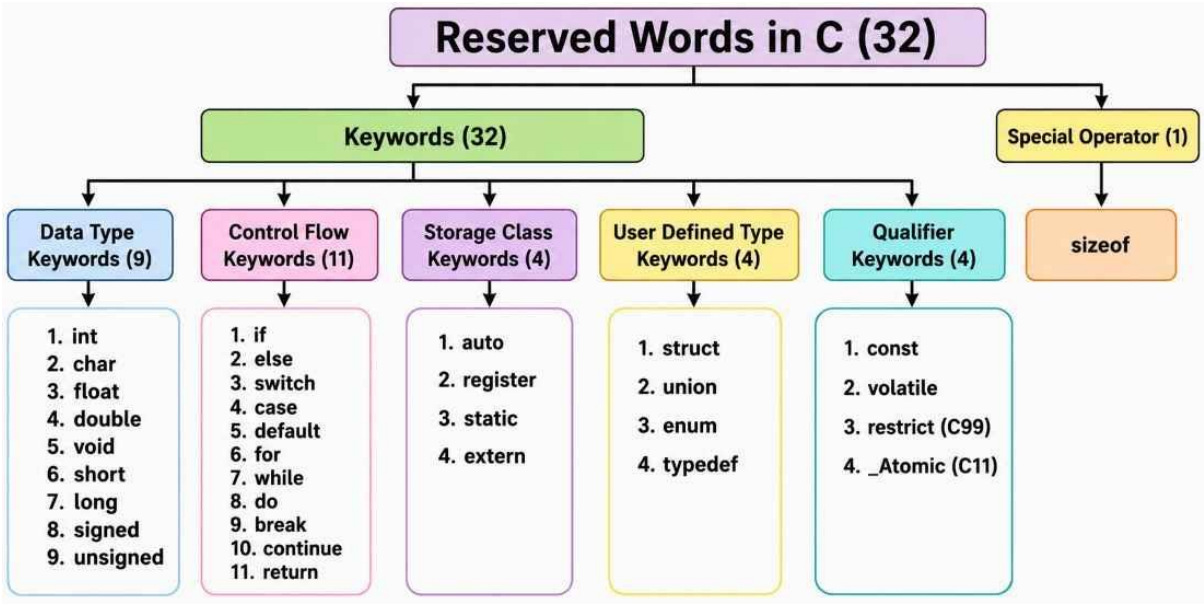
C Programming Tutorial

Your First C Program

C is a procedural language where execution starts from the main() function. Every statement ends with a semicolon ;, and code blocks are enclosed in curly braces {}. The language is compiled, meaning your code is transformed into machine code before execution. Let's begin with the classic "Hello, World!":

```
#include <stdio.h>  // This gives us input/output functions


int main() {
   printf("Hello, World!\n");
   return 0;  // 0 means "everything worked correctly"
}
```



Variables and Data Types

C has several basic data types. Unlike some languages, you must declare the type of every variable before using it:

```
#include <stdio.h>


int main() {
   // Integer types
int var;
```

```
// var is not initialised on declaration. It is given whatever value is in memory.
int age = 25;         // Whole numbers (typically 4 bytes)
   short small = 100;     // Smaller integer
   long big = 1000000L;    // Larger integer

   // Decimal numbers
   float price = 19.99f;   // Single precision (4 bytes)
   double pi = 3.14159;    // Double precision (8 bytes)

   // Character
   char grade = 'A';      // Single character in single quotes

   // Printing variables
   printf("Age: %d\n", age);
   printf("Price: %.2f\n", price);  // .2 shows 2 decimal places
   printf("Grade: %c\n", grade);
   printf("Pi: %f\n", pi);

   return 0;
}
```

Format specifiers: %d for integers, %f for floats/doubles, %c for characters, %s for strings.

Constants

You can make variables unchangeable using const:

```
#include <stdio.h>

int main() {
   const double TAX_RATE = 0.08;
   const int MAX_SCORE = 100;

   // TAX_RATE = 0.10;  // This would cause an error!
```

```
    printf("Tax rate: %.2f\n", TAX_RATE);
    printf("Max score: %d\n", MAX_SCORE);

    return 0;
}
```

Input from User

Use scanf() to read input from the keyboard:

```
#include <stdio.h>

int main() {
    int age;
    float salary;
    char initial;
    printf("Enter your age: ");
    scanf("%d", &age);  // & means "address of" - needed for scanf

    printf("Enter your salary: ");
    scanf("%f", &salary);

    printf("Enter your initial: ");
    scanf(" %c", &initial);  // Space before %c skips whitespace

    printf("\nYou entered:\n");
    printf("Age: %d\n", age);
    printf("Salary: %.2f\n", salary);
    printf("Initial: %c\n", initial);

    return 0;
}
```

Run on cpp.shell the "enter in" outputs are not printed until the end.

To fix and \n after the "Enter in..... \n"

Error in Visual Studio ..problem with scanf

```
#include <stdio.h>
int main() {
   int age;
   float salary;
   char initial;
   printf("Enter your age: ");
    if (scanf_s("%d", &age) != 1) {
      printf("Invalid input for age.\n");
      return 1;
   }
   printf("Enter your salary: ");
    if (scanf_s("%f", &salary) != 1) {
      printf("Invalid input for salary.\n");
      return 1;
   }
    printf("Enter your initial: ");

   if (scanf_s(" %c", &initial) != 1) {
      printf("Invalid input for initial.\n"); // Space before %c skips whitespace
      return 1;
   }
   printf("\nYou entered:\n");
   printf("Age: %d\n", age);
   printf("Salary: %.2f\n", salary);
   printf("Initial: %c\n", initial);
   return 0;
```

```
}
```

Arithmetic Operations

```
#include <stdio.h>

int main() {
   int a = 10, b = 3;

   printf("a = %d, b = %d\n", a, b);
   printf("Addition: %d\n", a + b);
   printf("Subtraction: %d\n", a - b);
   printf("Multiplication: %d\n", a * b);
   printf("Division: %d\n", a / b);     // Integer division (truncates)
   printf("Remainder: %d\n", a % b);     // Modulo operator

   // Using floats for precise division
   float x = 10.0, y = 3.0;
   printf("\nFloat division: %.2f\n", x / y);

   // Increment and decrement
   int count = 5;
   count++;  // count becomes 6
   printf("Count after increment: %d\n", count);
   count--;  // count becomes 5
   printf("Count after decrement: %d\n", count);

   // Compound assignment
   int num = 10;
   num += 5;   // num = num + 5
   printf("num after += 5: %d\n", num);
   num *= 2;   // num = num * 2
   printf("num after *= 2: %d\n", num);
```

```
    return 0;
}
```

Type Conversion

C automatically converts between types, but sometimes you need to control it:

```
#include <stdio.h>

int main() {
    int integer = 5;
    float floating = 2.5;

    // Automatic conversion (int becomes float)
    float result1 = integer + floating;
    printf("5 + 2.5 = %.2f\n", result1);

    // Integer division (truncates decimal)
    int result2 = 7 / 2;
    printf("7 / 2 = %d (truncated)\n", result2);

    // Explicit casting
    float result3 = (float)7 / 2;
    printf("(float)7 / 2 = %.2f\n", result3);

    // Converting float to int (truncates)
    int result4 = (int)3.9;
    printf("(int)3.9 = %d\n", result4);

    return 0;
}
```

Decision Making: if, else if, else

```
#include <stdio.h>
```

```
int main() {
     int number = 23;
   if (number >= 90) {
      printf("Number is greater than 90\n");
    }

   int score = 23;
   if (score >= 90) {
      printf("Grade: A - Excellent!\n");
    }
   else if (score >= 80) {
      printf("Grade: B - Good!\n");
    }
   else if (score >= 70) {
      printf("Grade: C - Satisfactory\n");
    }
   else if (score >= 60) {
      printf("Grade: D - Needs improvement\n");
    }
   else {
      printf("Grade: F - Failed\n");
    }
   // Using logical operators

   int age=10;



   if (age >= 18 && age <= 65) {
      printf("You are of working age.\n");
    }
   else if (age < 18 || age > 65) {
```

```
    printf("You are not of working age.\n");
  }

  return 0;

}
```

The switch Statement

Use switch for checking a variable against multiple specific values:

```
#include <stdio.h>

int main() {
  char grade;

  printf("Enter your grade (A, B, C, D, F): ");
  scanf(" %c", &grade);

  switch(grade) {
    case 'A':
    case 'a':
      printf("Excellent performance!\n");
      break;
    case 'B':
    case 'b':
      printf("Good job!\n");
      break;
    case 'C':
    case 'c':
      printf("Satisfactory\n");
      break;
    case 'D':
    case 'd':
```

```
            printf("You can do better\n");
            break;
        case 'F':
        case 'f':
            printf("Needs improvement\n");
            break;
        default:
            printf("Invalid grade entered\n");
    }

    return 0;
}
```

Loops: while and do-while

```
#include <stdio.h>

int main() {
    // While loop - checks condition first
    int count = 1;
    printf("While loop:\n");
    while (count <= 5) {
        printf("Count: %d\n", count);
        count++;
    }

    // Do-while loop - executes at least once
    int number=3;
    printf("\nDo-while loop \n");
    do {
        printf("number: %d\n", number);
         number--;
    } while (number != 0);
```

```
    // Infinite loop with break
    printf("\nLoop with break:\n");
    int i = 1;
    while (1) {  // Infinite loop
        printf("%d ", i);
        if (i == 5) {
            break;  // Exit loop
        }
        i++;
    }
    printf("\n");

    return 0;
}
```

Loops: for Loop

The for loop combines initialization, condition, and increment in one line:

```
#include <stdio.h>

int main() {
    // Basic for loop
    printf("Counting 1 to 10:\n");
    for (int i = 1; i <= 10; i++) {
        printf("%d ", i);
    }
    printf("\n");

    // Count backwards
    printf("\nCounting down:\n");
    for (int i = 10; i >= 1; i--) {
        printf("%d ", i);
```

```
    }
    printf("\n");

    // Skip even numbers with continue
    printf("\nOdd numbers only:\n");
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue;  // Skip rest of loop iteration
        }
        printf("%d ", i);
    }
    printf("\n");

    // Nested loops - multiplication table
    printf("\nMultiplication Table (5x5):\n");
    for (int row = 1; row <= 5; row++) {
        for (int col = 1; col <= 5; col++) {
            printf("%3d ", row * col);
        }
        printf("\n");  // New line after each row
    }

    return 0;
}
```

Arrays: Storing Multiple Values

Arrays store multiple values of the same type in contiguous memory. The index starts at 0:

```
#include <stdio.h>

int main() {
    // Declare and initialize an array
    int numbers[5] = {10, 20, 30, 40, 50};
```

```
// Accessing array elements
printf("First element: %d\n", numbers[0]);
printf("Third element: %d\n", numbers[2]);
printf("Fifth element: %d\n", numbers[4]);

// Modifying array elements
numbers[1] = 25;
printf("Modified second element: %d\n", numbers[1]);

// Using a loop to process array
printf("\nAll numbers:\n");
for (int i = 0; i < 5; i++) {
   printf("numbers[%d] = %d\n", i, numbers[i]);
}

// Calculate sum
int sum = 0;
for (int i = 0; i < 5; i++) {
   sum += numbers[i];
}
printf("Sum: %d\n", sum);

// Array with partial initialization
int scores[10] = {85, 92, 78};  // Rest are automatically 0
printf("\nScores array (first 5):\n");
for (int i = 0; i < 5; i++) {
   printf("scores[%d] = %d\n", i, scores[i]);
}

// Find maximum value
```

```
    int values[] = {45, 12, 89, 34, 67, 23};  // Size determined automatically
    int size = sizeof(values) / sizeof(values[0]);  // Calculate array size
    int max = values[0];

    for (int i = 1; i < size; i++) {
        if (values[i] > max) {
            max = values[i];
        }
    }
    printf("\nMaximum value: %d\n", max);

    return 0;
}
```

Multi-dimensional Arrays

```
#include <stdio.h>

int main() {
    // 2D array (3 rows, 4 columns)
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    printf("Matrix:\n");
    for (int row = 0; row < 3; row++) {
        for (int col = 0; col < 4; col++) {
            printf("%3d ", matrix[row][col]);
        }
        printf("\n");
    }
```

```
    // Create a simple grade book
    int grades[4][3] = {
        {85, 90, 78},  // Student 0
        {88, 92, 84},  // Student 1
        {95, 87, 91},  // Student 2
        {76, 82, 79}   // Student 3
    };

    printf("\nGrade Book:\n");
    printf("Student\tExam1\tExam2\tExam3\tAverage\n");
    for (int student = 0; student < 4; student++) {
        int total = 0;
        printf("%d\t", student + 1);
        for (int exam = 0; exam < 3; exam++) {
            printf("%d\t", grades[student][exam]);
            total += grades[student][exam];
        }
        printf("%.1f\n", total / 3.0);
    }

    return 0;
}
```

What a String Really Is

In C, there is no separate "string" type. A string is just an array of characters, and the only thing that makes it a "string" is that it ends with a special character called the null terminator, written `'\0'`.

So when you write:

```
char name[20] = "Alice";
```

The array actually contains `'A'`, `'l'`, `'i'`, `'c'`, `'e'`, `'\0'`, and then leftover unused space up to index 19. The compiler adds the `'\0'` for you automatically when you use a string literal in quotes.

If you build the array manually, like:

```
char city[30] = {'N', 'e', 'w', ' ', 'Y', 'o', 'r', 'k', '\0'};
```

you have to add the `'\0'` yourself. Forget it, and it's not a string — it's just an array of characters, and `printf("%s")` will keep reading past the end into random memory until it happens to hit a zero byte somewhere.

Why the Null Terminator Matters

Every `%s` in `printf` works the same way: start at index 0, keep going character by character, and stop when you hit `'\0'`. That's the entire mechanism. There is no stored length anywhere. The `'\0'` *is* the length marker.

Doing Things Manually

Example

Finding the length: start at index 0, increment a counter while the character is not `'\0'`.

```
int len = 0;
while (name[len] != '\0') {
    len++;
}
```

Functions - Organizing Your Code

Functions help break down complex programs into manageable pieces:

#include <stdio.h>

int add(int a, int b) {

  return a + b;

```
}

void printGreeting() {
    printf("Hello! Welcome to C programming.\n");
}

double calculateAverage(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    return (double)sum / size;
}

// Main function
int main() {
    // Function calls
    int sum = add(5, 3);
    printf("5 + 3 = %d\n", sum);

    printGreeting();

    int scores[] = {85, 90, 78, 92, 88};
    int size = sizeof(scores) / sizeof(scores[0]);
    double avg = calculateAverage(scores, size);
    printf("Average score: %.2f\n", avg);

    // Functions can be called multiple times
    printf("10 + 20 = %d\n", add(10, 20));
    printf("100 + 200 = %d\n", add(100, 200));
```

```
    return 0;
}
```

Passing Arrays to Functions

When you pass an **array** to a function, the function does **not** get its own copy of the array — it works directly with the **original array** from the caller.

Any change the function makes to the array's elements will be visible in the original array back in `main`.

Example: A function that modifies an array

```c
#include <stdio.h>

void doubleAll(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = arr[i] * 2;   // this change affects the original array
    }
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Before: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    doubleAll(numbers, size);   // pass the array
```

```
    printf("After:  ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    return 0;
}
```

Output:

```
text

Before: 1 2 3 4 5
After:  2 4 6 8 10
```

Notice that `numbers` in `main` was changed — even though we never touched `numbers` directly after calling the function.

This is very useful: it lets functions like `calculateAverage` **read** an array, and functions like `doubleAll` **update** an array, without returning anything.

Pass by Value

For ordinary variables (like `int`, `double`, `char`), C uses **pass by value**. This means the function receives a **copy** of the value. Whatever the function does to its parameter has **no effect** on the original variable in the caller.

```
#include <stdio.h>

void tryToChange(int x) {
    x = 999;              // only changes the local copy
    printf("Inside function: x = %d\n", x);
}

int main() {
    int value = 5;

    printf("Before call: value = %d\n", value);
    tryToChange(value);
```

```
    printf("After call:  value = %d\n", value);

    return 0;
}
```

Output:

```
Before call: value = 5
Inside function: x = 999
After call:  value = 5
```

Even though the function set `x = 999`, `value` in `main` is still 5. The function worked on a copy.

This is why `add(5, 3)` works safely — `add` gets copies of 5 and 3 and cannot accidentally change anything in `main`.

Pass by Reference

Pass by reference means the function does **not** get a copy — it gets a **reference** to the caller's actual variable. The parameter *is* the original variable under a different name. Any change the function makes is directly a change to the caller's variable.

Note: The code at the top of this page is C, which does **not** have an & reference syntax. The examples below are in C++ so you can see what pass by reference looks like explicitly. There are a number of functions in Lab 1 that contain a pass by reference.

In C++, you write pass by reference with the & symbol in the parameter list:

```
void tryToChange(int &x) {   // note the & — pass by reference
    x = 999;                 // this changes the caller's variable
}
```

Now if you call `tryToChange(value)`, `value` really does become 999.

Example: Value vs. Reference side by side

```
#include <stdio.h>



// Pass by value — works on a copy

 void addOneByValue(int x) {
```

```
    x = x + 1;
    printf("Inside addOneByValue: x = %d\n", x);
}
  // Pass by reference — works on the original
void addOneByReference(int& x) {
    x = x + 1;
    printf("Inside addOneByReference: x = %d\n", x);
}

int main() {
    int a = 5;
     printf("a before    a = %d\n", a);
    addOneByValue(a);
    printf("After addOneByValue:    a = %d\n", a);
    addOneByReference(a);
    printf("After addOneByReference: a = %d\n", a);
    return 0;
}
```

Output:

a before a = 5

Inside addOneByValue: x = 6

After addOneByValue: a = 5

Inside addOneByReference: x = 6

After addOneByReference: a = 6

Notice the difference:

- `a` was passed by **value**, so it stayed `5` in `main`.
- a was passed by **reference**, so it became `6` in `main` — the function changed the original.

Why Use Pass by Reference?

1. **Modify the caller's variable.** A function like `swap(int &a, int &b)` can actually exchange two variables. With pass by value, that would be impossible.

```cpp
void swap(int &a, int &b) {
    int temp = a;
    a = b;
    b = temp;
}
```

2. **Avoid copying large data.** Passing a big object by value copies the whole thing. Passing by reference avoids that cost. We will see this when passing objects.
3. **Return multiple values.** A function can only `return` one value, but with references it can change several caller variables at once.

Value vs. Reference Comparison

Feature	Pass by Value	Pass by Reference
What is passed	A **copy** of the value	A **reference** to the original variable
Syntax (C++)	`void f(int x)`	`void f(int &x)`
Can the function change the caller's variable?	**No**	**Yes**
Typical use	When the function only needs to read the value	When the function needs to modify the caller's variable, or avoid copying large data